

Simulasi Agen Koordinator Truk Sampah Menggunakan Matheuristic Rollout dalam Kondisi Uncertain (Stochastic) Kota Balikpapan

Kelompok 11

Nama	NIM
Muhammad Zaini	11241064
Ahmad Rafif Rafi	11241006
Mayoga Finanda	11241044
Ferdianta Tarigan	11241030

A. Abstract

Pengangkutan sampah di Kota Balikpapan menghadapi berbagai banyak tantangan karena kondisi yang dinamis dan sulit diprediksi, jumlah sampah di setiap Tempat Penampungan Sementara (TPS), kondisi lalu lintas, serta keterbatasan jumlah armada truk. Permasalahan tersebut sering mengakibatkan jarak tempuh yang tidak efisien, waktu kerja yang melebihi jam operasional, ketimpangan beban kerja antar truk, dan masih adanya TPS yang belum terlayani. Oleh karena itu, diperlukan sistem pengelolaan yang mampu mendukung pengambilan keputusan secara efektif.

Tugas ini mengembangkan sebuah agen AI yang berperan sebagai koordinator pengangkutan sampah dalam sebuah simulasi yang merepresentasikan kondisi nyata di lapangan. Agen AI tersebut menggunakan pendekatan matheuristic rollout untuk mengevaluasi berbagai alternatif keputusan secara bertahap dan memilih keputusan terbaik berdasarkan fungsi tujuan. Fungsi tujuan dirancang dengan mempertimbangkan empat aspek utama, yaitu meminimalkan jarak tempuh truk, mengurangi waktu lembur, memastikan seluruh TPS terlayani, serta menjaga keseimbangan beban kerja antar truk, dengan prioritas utama pada pelayanan TPS.

Hasil simulasi menunjukkan bahwa pendekatan ini mampu meningkatkan kinerja dan efisiensi proses pengangkutan sampah. Jarak tempuh dan waktu kerja truk dapat dikendalikan dengan lebih baik, beban kerja antar truk menjadi lebih seimbang, serta jumlah TPS yang tidak terlayani dapat dikurangi. Dengan demikian, agen AI yang dikembangkan diharapkan dapat menjadi solusi pendukung dalam pengelolaan pengangkutan sampah di Kota Balikpapan agar lebih efisien dan mudah diterapkan di lapangan.

B. Data Used

1. Total TPS, sebaran, dan jumlah sampah per hari

Data diambil dari KAJIAN POOL KENDARAAN PENGANGKUTAN SAMPAH DI KOTA BALIKPAPAN TAHUN 2022

NOTE: Dikarenakan terdapat 73 TPS, kami menghomogenkan seluruh data dengan mengambil rata-rata dari seluruh TPS dan memberikannya interval $\pm 30\%$ randomisasi agar dapat lebih fokus ke proses pengembangan AI.

2. Jadwal pengambilan sampah

Jadwal Angkut Sampah Balikpapan

3. Lokasi TPA

Peresmian TPA Manggar

4. Jumlah Truk

C. Methods

1. Matheuristic Rollout

Metode rollout dengan teknik matheuristic yang bertujuan untuk meningkatkan kualitas keputusan dalam masalah sequential decision-making di lingkungan yang stochastic. Rollout ini bekerja dengan memanfaatkan base policy atau keputusan cepat dan sederhana sebagai kebijakan awal, lalu melakukan evaluasi ke depan (look-ahead) untuk memilih tindakan yang lebih baik daripada keputusan dasar sebelumnya. Setiap aksi yang dihitung nilai minimum berdasarkan objective function yang ada kemudian aksi dengan nilai minimum tersebut akan di eksekusi langsung.

Peningkatan keputusan terjadi karena Rollout tidak hanya berusaha melihat kondisi saat ini, tetapi juga memprediksi dampak tindakan pada beberapa langkah ke depan terutama pada kondisi yang berubah-ubah dari fluktuasi volume sampah yang tidak selalu diketahui

Kemudian di tahap eksekusi rute, sistem menggunakan `shortest path` dari `OSMnx` untuk menentukan jalur perjalanan truk pada jaringan jalan yang dapat diberikan pengecualian. Namun, aspek ini tidak menjadi fokus utama penelitian karena pathfinding hanya berfungsi sebagai komponen teknis pendukung.

Fokus utama simulasi adalah pada multi-target, multi-instance decision-making, yaitu bagaimana agen truk mengambil keputusan rute dan prioritas TPS secara adaptif melalui mekanisme Rollout dan tidak melakukan tabrakan tugas kecuali dibutuhkan.

2. Objective Function

Untuk setiap TPS (t), skor dihitung sebagai:

$$\text{Skor}(t) = W_d \cdot \frac{1}{d(t) + 100} + W_g \cdot \frac{g(t)}{1000}$$

- $d(t)$: jarak kendaraan ke TPS
- $g(t)$: jumlah sampah di TPS
- W_d : bobot jarak (DISTANCE_WEIGHT)
- W_g : bobot sampah (GARBAGE_WEIGHT)
- TPS dengan skor terbesar akan dipilih
- TPS dengan sampah ≤ 10 kg tidak dipertimbangkan
- TPS yang terlalu jauh atau tidak memiliki jalur aman diabaikan

Hal ini bertujuan untuk memaksimalkan jumlah pengambilan sampah dengan jarak tempuh sekecil mungkin

3. Knowledge model

a) **Peta dan Lokasi TPS/TPA Diketahui** Representasi peta kota:

$G = (V, E)$

Makna variabel:

Variabel	Penjelasan
G	Graph atau peta kota
V	Himpunan semua node (simpul jalan, TPS, TPA, garasi)
E	Himpunan semua edge (ruas jalan)
(x_v, y_v)	Koordinat geografis dari node v
V_{TPS}	Subset node yang merupakan TPS
V_{TPA}	Subset node yang merupakan TPA
V_{GARAGE}	Subset node yang merupakan GARAGE

Formulasi:

$\forall v \in V$: lokasi (x_v, y_v) diketahui
 $v \in V_{\text{TPS}}$ (lokasi TPS diketahui)
 $v \in V_{\text{TPA}}$ (lokasi TPA diketahui)
 $v \in V_{\text{GARAGE}}$ (lokasi Garasi diketahui)

Artinya model sudah mengetahui semua rute serta lokasi TPS, TPA, dan Garasi secara lengkap.

b) Volume Sampah TPS Tidak Diketahui Sebelum Truk Tiba Model volume sampah:

$$W_i \sim g_i(t)$$

Makna variabel:

Variabel	Penjelasan $\hat{W}_i(t)$
W_i	Volume sampah aktual di TPS i
$g_i(t)$	Distribusi probabilitas volume sampah TPS i
$A_i(t)$	Indikator apakah sudah ada truk yang tiba di TPS i
$\hat{W}_i(t)$	Informasi volume sampah yang diketahui sistem

Indikator kunjungan:

$$A_i(t) = 0 \text{ (belum ada truk yang tiba)}$$

$$A_i(t) = 1 \text{ (sudah ada truk yang tiba)}$$

Pengetahuan sistem:

$$\hat{W}_i(t) = W_i(t) \quad \text{jika } A_i(t) = 1$$

$$\hat{W}_i(t) = \text{unknown} \quad \text{jika } A_i(t) = 0$$

Artinya model tidak mengetahui berapa volume sampah sebelum setidaknya satu truk benar-benar tiba di TPS tersebut.

D. Implementation

1. Environment Setup

Pada sistem yang telah kami buat, kami menggunakan library pygame untuk melakukan visualisasi dan simulasi untuk environment. Dengan data .graphml yang diambil langsung dari openstreetmap.org melalui library osmnx yang tersedia di python.

Untuk menambahkan data TPA, TPS, Garasi, Mobil, dan Kemacetan, kami menyediakan map editor sehingga pengeditan dapat dilakukan dengan lebih leluasa dan mudah yang datanya akan disimpan dalam file .json. Meskipun

begitu, data yang cukup jarang diubah seperti kecepatan kendaraan, waktu shift, serta warna warna node dan edges, kami letakkan di `src/environment.py`.

2. Data Modeling

a) Nodes data model

```
node_id: {
    "tps": boolean,
    "tpa": boolean,
    "garage": boolean,

    "tps_data": {
        "nama": String,
        "sampah_kg": float,
        "sampah_per_hari": float, # Interval  $\pm 30\%$ 
    },

    "tpa_data": {
        "nama": String,
        "total_sampah": float
    },

    "garage_data": {
        "nama": "Garage",
        "total_armada": int,
        "armada_bertugas": int,
        "armada_standby": int
    }
}
```

b) Truck data model

```
truck_id: {
    "garage_node": node_id,
    "state": enum CarState, # Idle, Moving, Loading, Unloading, Stuck, Standby
    "speed": float,
    "daily_dist": float,
    "total_dist": float,
    "load": float,
    "max_load": float,
    "route": list[node_id]
}
```

c) Environment

```

# ===== WINDOW =====
APP_NAME = "Simulasi Truk Sampah Balikpapan"
WIDTH = 1000
HEIGHT = 600
CAM_SPEED = 10
MAX_FPS = 60

# ===== TEST SETUP =====
GRAPH_FILE = "./data/simpl_balikpapan_timur_drive.graphml"

# ===== VEHICLE =====
VEHICLE_SPEED = 60
VEHICLE_CAP = 30000

# ===== SHIFT SETTINGS (00:00 WITH INTEGER 0) =====
SIM_START = 1
SHIFT_START = 1
SHIFT_END = 13
TIME_OFFSET = 17

# ===== SPRITES =====
NODE_COL = (255,120,120) # Node kuning
LINE_COL = (150,150,150) # Jalanan putih

AGENT_COL = (0,255,0) # Mobil Hijau
TPS_COL = (255,220,0) # TPS kuning
TPA_COL = (0,150,255) # TPA biru
GARAGE_COL = (139, 69, 19) # Garasi coklat

```

3. Rollout Algorithm Integration

a) **Dispatch** Pada awal jam kerja, truk yang tersedia akan dikeluarkan dari garasi dan pergi menuju TPS yang tersedia, pada proses ini dilakukan base policy untuk menentukan rute tercepat secara deterministik, kemudian dilakukan proses look ahead berdasarkan pengalaman dari truk sebelumnya (jumlah sampah terakhir).

b) **Gathering** Setelah truk mencapai TPS, sampah akan diambil berdasarkan kapasitas maksimal truk, kemudian diantarkan ke TPA, proses ini akan dilakukan selama jam kerja dan akan diakhiri jika jam kerja selesai ataupun semua sampah harian di TPS telah dikosongkan.

c) **Reschedule/Rerouting** Jika truk mendapat pengetahuan tentang jumlah sampah di TPS, modal akan melakukan reschedule ulang maupun stealing untuk

memastikan apakah rute sekarang masih efektif atau tidak dan menyesuaikan dengan meminimalkan perubahan rute pada truk lain yang tidak relevan dengan kondisi yang terjadi.

d) Ending Jika sampah sudah habis atau jam kerja sudah selesai, semua truk akan kembali dan model akan melakukan evaluasi untuk menentukan mekanisme saat dispatch di jam kerja selanjutnya agar lebih efektif.

4. Simulation Engine

a). Folder Structure

```

root/
├─ data/           # Dataset AI
├─ scrapper/       # Script untuk mengambil dataset graphml
├─ src/           # Program utama untuk simulasi (pygame)
│   └─ classes/
│       ├── ai_model.py      # AIModel dimana semua keputusan diambil
│       ├── knowledge.py     # KnowledgeModel tempat menyimpan informasi yang dimiliki
│       └─ vehicle.py        # Actuator dan Sensor dari truk sampah
│   └─ testing/
│       └─ benchmark.py      # Mengetes benchmark
│   └─ utils/
│       ├── controls.py      # Controller kamera di simulation
│       ├── nodes.py         # Node Generator berdasarkan data yang ada
│       ├── shared.py        # Tempat menyimpan data runtime
│       ├── timesync.py      # Time accelerator dan jam simulasi
│       └─ viewer.py         # Renderer simulation
├─ environment.py  # Data data environment dasar seperti ukuran window, truk, jam aktif, dan warna
├─ simulation.py   # Simulation runner
└─ start.py        # Full Runner

├─ window/        # Window Tkinter untuk map editor
├─ Report.md      # Laporan ini
└─ README.md      # Dokumentasi utama proyek

```

b). AIModel Implementation Flow 1) Main Loop

Bagian ini akan dipanggil tiap frame atau iterasi dari simulasi yang akan mengambil keputusan tiap decision interval, ini juga menyebabkan hasil simulasi tidak efektif di kecepatan tinggi karena perogram tidak sempat "berpikir" untuk rencana selanjutnya

```

def update(self, dt, vehicles):
    if self.shared.paused:
        return

```

```

self.last_decision_time += dt
if self.last_decision_time >= self.decision_interval:
    self.last_decision_time = 0
    self.make_decisions(vehicles)

```

2) Decision Making

Fase AI akan dikontrol berdasarkan jam simulasi dan dipisah dalam 3 tahap, yaitu DISPATCH, GATHERING, dan ENDING

```

def make_decisions(self, vehicles):
    current_hour = self.shared.sim_hour

    if current_hour >= self.SHIFT_START and not self.dispatch_done:
        self.phase_dispatch(vehicles)
        self.current_phase = "GATHERING"
        self.dispatch_done = True
        return

    if current_hour >= (self.SHIFT_END - self.OVERTIME_BUFFER):
        if self.current_phase != "ENDING":
            self.phase_ending(vehicles)
            self.current_phase = "ENDING"
        return

    if self.current_phase == "GATHERING":
        self.phase_gathering(vehicles)

```

3) Dispatch Phase

Pada fase ini AI akan dikeluarkan dari Garasi saat SHIFT_START``, disini juga digunakan fungsi `_find_next_tps` untuk melakukan scoring dan lookahead beberapa langkah kedepan

```

def phase_dispatch(self, vehicles):
    idle_vehicles = [v for v in vehicles if v.state.lower() == "idle"]

    for vehicle in idle_vehicles:
        next_tps = self._find_next_tps(vehicle)
        if not next_tps:
            continue

        path = self._safe_path(vehicle.current, next_tps, vehicle.G)
        task = {"type": "collect", "tps_id": next_tps, "assigned_at": current_time}
        self._assign_task(vehicle, task)
        vehicle.set_path(path)
        vehicle.state = "to_tps"

```

Scoring dilakukan berdasarkan jarak dan jumlah sampah (jika sudah diketahui)


```
distance_score = 10000.0 / (my_distance + 100)
garbage_score = garbage / 1000.0
```

```
score = (
    self.DISTANCE_WEIGHT * distance_score +
    self.GARBAGE_WEIGHT * garbage_score
)
```

4) Gathering Phase

Pada tahap ini, kendaraan akan mengambil sampah di TPS sembari melihat kapasitas truk yang tersisa, jika truk sudah benar benar penuh, maka kendaraan akan dipaksa pergi ke TPA untuk unload muatan, pada bagian ini juga digunakan look ahead untuk menentukan pilihan selanjutnya.

```
def _handle_at_tps(self, vehicle):
    if vehicle.actuator_is_full():
        self._route_to_tpa(vehicle)
        return

    loaded = vehicle.actuator_load_from_tps()
    if vehicle.actuator_is_full():
        self._route_to_tpa(vehicle)
        return

    next_tps = self._find_next_tps(vehicle)
    if next_tps:
        path = self._safe_path(vehicle.current, next_tps, vehicle.G)
        task = {"type": "collect", "tps_id": next_tps, "assigned_at": current_time}
        self._assign_task(vehicle, task)
        vehicle.set_path(path)
        vehicle.state = "to_tps"
    else:
        self._route_to_tpa(vehicle)
```

Untuk mencegah truk idle mengambil tugas yang sangat jauh, maka diterapkan mekanisme STEAL, atau AI akan mengarahkan truk terdekat yang tersedia untuk "Mencuri" target dari truk yang jauh tersebut selama jaraknya masih masuk akal.

```
if (my_distance < assigned_dist * self.STEAL_DISTANCE_RATIO and
    distance_advantage >= self.STEAL_MIN_ADVANTAGE):
    self._cancel_assignment(assigned_vehicle, tps_id)
    can_take = True
```

Untuk masalah routing disederhanakan menggunakan nx.shortest_path

```
def _safe_path(self, start, end, G):
    try:
```

```

        return nx.shortest_path(G, start, end, weight="length")
    except Exception:
        return None

```

5) Ending Phase

Kendaraan yang sudah menyelesaikan shift akan diarahkan kembali ke garasi selama mereka tidak membawa sampah lagi, jika tidak, mereka harus membuang sampah tersebut ke TPA sebelum kembali.

```

def phase_ending(self, vehicles):
    for vehicle in vehicles:
        if vehicle.load > 0:
            self._force_to_tpa()
        else:
            self._route_to_garage(vehicle)

```

E. Demo

1. Cara Menjalankan Simulasi

a) Cek apakah pip sudah terpasang:

```
python -m pip --version
```

b) Masuk ke virtual environment:

Windows (PowerShell)

```
python -m venv venv
venv\Scripts\activate
```

Linux (bash)

```
python3 -m venv venv
source venv/bin/activate
```

c) Setelah environment aktif, jalankan kode berikut untuk menginstall library yang dibutuhkan:

```
pip install numpy pandas scikit-learn matplotlib pygame osmnx
```

NOTE: Untuk Linux ubuntu, gunakan pygame-ce

d) Kemudian ketik ini di terminal untuk menjalankan simulasi:

```
python -m src.start
```

2. Illustration

<https://github.com/user-attachments/assets/b3aa68ec-3a79-4587-9721-16b73aa5100a>

3. Result

SIMULATION SUMMARY

```
=====
Days Simulated:          1
Simulation Time:         71.85s

Total Distance:          320.13 km
Total Garbage Collected: 67,941 kg
Collection Rate:         100.0%
Efficiency:              212.2 kg/km
Vehicle Utilization:     100.0%
TPS Coverage:            100.0%
=====
```

```
Quick test complete!
Garbage: 67940.54 kg
Trips: 0
Time: 71.85s
```

F. Summary

Pada simulasi dan benchmark yang kami buat, meskipun lokasi serta data hanya mendekati dan tidak 100% sama dengan kondisi nyata. Program ini dapat mempermudah menentukan apakah sebuah rute pengambilan sampah sudah efisien atau tidak dengan cepat dan mudah jikalau ada perubahan kondisi seperti penempatan TPS atau TPA baru serta peningkatan jumlah kendaraan karena simulasi menyiapkan cara yang mudah untuk mengedit environment jika dibutuhkan. Selain itu, simulasi ini juga dapat dilakukan tidak hanya di balikpapan, namun juga dapat menggunakan rute kota lain jika dibutuhkan dikarenakan adanya scrapper map yang dapat dengan mudah digunakan.

G. References

- Anuar, W. K., Lee, L. S., Seow, H.-V., & Pickl, S. (2021). A Multi-Depot Vehicle Routing Problem with Stochastic Road Capacity and Reduced

Two-Stage Stochastic Integer Linear Programming Models for Rollout Algorithm. *Mathematics*, 9(13), 1572. <https://doi.org/10.3390/math9131572>.

- Pemerintah Kota Balikpapan Dinas Lingkungan Hidup. (2022). Kajian pool kendaraan pengangkutan sampah di Kota Balikpapan, Tahun 2022 (Laporan Akhir). Pemerintah Kota Balikpapan Dinas Lingkungan Hidup.,
- RRI. (2024, Januari 31). DLH Balikpapan atur ulang jadwal angkut sampah. rri.co.id. Diakses 13 Desember 2025, dari <https://rri.co.id/lain-lain/537534/dlh-balikpapan-atur-ulang-jadwal-angkut-sampah>
- Kementerian Pekerjaan Umum dan Perumahan Rakyat. (2019, Desember 19). Diresmikan Presiden, pembangunan TPA Sampah Manggar di Balikpapan terbaik di Indonesia. pu.go.id. Diakses 13 Desember 2025, dari <https://pu.go.id/berita/diresmikan-presiden-pembangunan-tpa-sampah-manggar-di-balikpapan-terbaik-di-indonesia>
- Hidayat, R.. Kajian Pengangkutan Sampah di Kota Balikpapan. Diperoleh dari https://repository.itk.ac.id/17715/16/13171008_presentation.pdf
- GITHUB: <https://github.com/muhammadzaini213/AI-Pemberantas>
- REPORT: <https://github.com/muhammadzaini213/AI-Pemberantas/blob/main/Report.md>
- GOOGLE COLAB: https://colab.research.google.com/drive/1eFsy2d_rU2F5ibHSfCYxPmlWD4sY0hNm?usp=sharing